DEVOPS & CLOUD [SWE] · SCALER ACADEMY

$ cat notes.md

DevOps & Cloud [SWE] — Class 3 Notes

LECTURE TITLE	**Shell Scripting**
DATE/TIME	**18 August 2026, 09:30 AM**
DURATION	**120 minutes**
INSTRUCTOR	**Nensi Ravaliya**
BIG IDEA	**A shell script is just the commands you already know, saved in a file, so you never have to type them again.**
SOURCES	**this class's transcript + the scripts in `Class-03/` (written live in class) + the instructor's official course repo Nency-Ravaliya/devops-heros → `session3-shell-scripting/`**

WHAT'S INSIDE — 17 SECTIONS

00 ⚡ The 60-Second Version
01 🗓️ Logistics & Warm-Up
02 🐚 Why Learn Shell Scripting At All?
03 ☕ Warm-Up Game — The Command Relay
04 🐚 Your First Shell Script — hello.sh
05 📝 Assignment 1 — Overwrite AND Append
06 🔤 Variables
07 ⌨️ Runtime Input — read -p
08 🚦 Conditions — if / elif / else
09 🔁 Loops
10 🧩 Functions
11 📝 The Big Combined Assignment
12 🟫 Live Review of Submitted Scripts
13 👋 Wrap-Up
14 📔 Cheat-Sheet — Everything From Today
15 📝 Homework Round-Up
16 🗒️ Notes on the Transcript Itself

Rebuilt from the raw class transcript — nothing cut, just organised, de-noised and made readable. `theme: Terminal Green`

00 ⚡ The 60-Second Version

1. **Shell scripting = Linux commands + programming.** Variables, if/else, loops, functions — bolted onto `mkdir`, `cat`, `ps`.
2. **Why you'll actually use it:** a real DevOps job hands you *"go fetch the logs off these servers and dump them into S3 / Grafana."* You do that with a script on a cron job.
3. **First line of every script:** `#!/bin/bash` — the shebang. It says "run me with Bash."
4. **A fresh script won't run.** You get `Permission denied` until you `chmod +x` it.
5. **Two ways to run one:** `./script.sh` or `sh script.sh`.
6. `>` **overwrites,** `>>` **appends.** One character, very different day.
7. `variable=value` **— NO SPACES around the** `=`**.** This is the #1 beginner error.
8. **To capture a command's output into a variable, you need** `$( )`**:** `current_date=$(date)`. Just writing `echo $date` prints nothing.

◆ ◆ ◆

01 🗓️ Logistics & Warm-Up

Monday-morning energy

🎤 SAID IN CLASS

"Good morning. Morning is good or not? My morning is awesome, by the way — even though it's not Monday."

She'd started driving at 8 AM *"full speed"*, then hit **Electronic City traffic** and spent the drive wondering if she'd make it.

Admin bits

Topic	Outcome
Online vs offline	Confirmed for this batch (**Section A**) — offline attendance is **mandatory**
The missing Friday class	She showed up 5 minutes late, found an empty room, then discovered it was a **schedule mismatch** — no class was actually scheduled. *"That was good."* 😄
The make-up class	Scheduled for the next day, ~4:45–5:45 PM — but for **Batch B**, not this one
Batch A vs B	*"I don't know this batch A, batch B. I just know how to teach and how to take attendance. That's it."* 😄

Topic	Outcome
Notes on GitHub	Asked if anyone had read the session MD file. She admitted she **hadn't uploaded it yet** — and did it live, during class

The honesty poll

🎤 SAID IN CLASS

"How many of you practised the Linux commands? Okay — be honest. If not, then okay, fine."

And user/group management? *"Have you created a user, a group, anything?"* — mixed hands. Several hadn't practised.

Weekend chat

Asked about everyone's weekend and exams. Verdict from the room: exams are **hell** — but right after, *"we can go outside, we can hang out, we can have some pani puri, we can go on a long ride."* Before the exam though? Hell. 😄

📚 Resources she shared (3 PDFs)

#	PDF	What's in it
1	**Basic Linux commands**	the Class 2 material, for revision
2	**Advanced Linux commands**	the deeper set
3	**Cheat sheet**	every command + what it does, side by side

💡 GOOD TO KNOW

Her advice: *"Check the first one for basic commands, and the other one for advanced commands if you have some time."* And before anything else today: **refresh every command from last session** — `ls`, `chmod`, the user-management set — because today builds directly on them.

🔧 Technology, again

Two things broke live:

1. She couldn't log into the AWS console — an **MFA / authenticator** problem on her account. *"I am not able to login in AWS console."*
2. Her **mouse and hover stopped working** mid-class. *"Bad issues. I guess it needs my mouse."*

The workaround: she abandoned EC2 and did the whole class in her **local terminal + VS Code**.

😄 CLASS MOMENT

Her verdict afterwards: *"Next time I will not use EC2, surely — this is way better than using EC2."* Sometimes local is just faster.

◆ ◆ ◆

02 🐚 Why Learn Shell Scripting At All?

✅ LOCK THIS IN

The one-line definition from class: *"Shell scripting is nothing but you are adding the programming knowledge"* — loops, variables, arrays, if-else conditions — *"we're just using the programming language with the Linux commands."*

The actual job you'll be given

🧠 PICTURE THIS

Picture this — a Monday at your first DevOps job. Your lead says: *"the logs for these 20 servers are piling up in `/var/log` and `/var/www/html/logs`. Every night, collect them, and drop them into our S3 bucket so Grafana can chart them."*

You are not going to SSH into 20 servers every night at midnight. You write a **script**, and you put it on a **cron job**.

The pipeline she described:

```
server's log folders  (/var/log, /var/www/html/logs)
        ↓   fetched by a shell script
     cron job  (runs it automatically, on schedule)
        ↓
  storage:  AWS S3  /  Azure Storage  /  GCP Storage
        ↓   or straight to
  monitoring:  Grafana dashboard
```

✅ LOCK THIS IN

What "enough" looks like: *"For those cases you need to learn at least basic-to-intermediate shell scripting"* — variables, if/else, loops, and functions. That's the whole shopping list, and it's what today covers.

💡 GOOD TO KNOW

The cron job she keeps mentioning — here's what one actually looks like, since it's the reason the whole class exists.

`cron` is Linux's alarm clock. You give it **five numbers and a command**:

```
┌──────── minute        (0–59)
│ ┌────── hour          (0–23)
│ │ ┌──── day of month  (1–31)
│ │ │ ┌── month         (1–12)
│ │ │ │ ┌ day of week   (0–6, Sunday = 0)
│ │ │ │ │
0 2 * * *   /home/ubuntu/fetch-logs.sh
└──┬──┘ └──────────┬──────────┘
   │               │
 "every day      "run this script"
  at 02:00"
```

`*` means "every". So `0 2 * * *` = minute 0, hour 2, every day, every month, every weekday → **2 AM daily.**

```
*/5 * * * *    every 5 minutes
0 * * * *      every hour, on the hour
0 0 * * 0      midnight every Sunday
```

Install it with:

```
crontab -e     # opens your personal schedule in an editor
crontab -l     # list what's scheduled
```

✅ **So the full picture of the job she described:** you write `fetch-logs.sh` today, and one `crontab` line makes it run at 2 AM forever without you being awake. **That's the entire value of shell scripting in one sentence.**

She also promised a separate PDF specifically on **log-related commands**.

◆ ◆ ◆

03 ☕ Warm-Up Game — The Command Relay

She opened with *"Who is the Harry Potter fan?"* as an icebreaker, then set up a round-robin quiz.

The rules: she asks nothing. Instead —

```
Team B asks Team A  →  Team A asks Team C  →  Team C asks Team B  →  repeat
```

Only Linux commands from the previous session were allowed. Laptops out, but…

> 😄 **CLASS MOMENT**
>
> *"Don't read Google. Don't read ChatGPT. Tell me."*

Getting the first volunteer took a while: *"Anyone? Are you sleeping? You had your breakfast, you had your tea, coffee…"* 😄

The questions that actually got asked

#	Question	Asked → Answering	The answer
1	What is `ll` ?	B → A	The **long-listing** form of `ls` . She had to correct the room twice — *"`ll` , right? Not `ln` ."* Someone kept hearing "link".
2	Command to display **disk usage**?	A → C	`df -h` — the `-h` for human-readable format
3	What does `ps` do?	C → B	Lists running processes. *"Nice, good."*
4	What is the `man` command for?	B → A	Displays a command's **manual**. She pushed for more: *"Explain to me how you're going to use it, for which purpose — is it like a help?"*
5	Difference between `cp` and `mv` ?	A → C	`cp` = copy-and-paste · `mv` = **cut**-and-paste. *"Exactly, this is the word."*
6	Which command to **search text**?	→ B	`grep`
7	Exact difference between `man` and `help` ?	B → A	`man` opens the full **manual page**; `help` is for **shell built-ins**. This one got muddled in the live back-and-forth — see the clarification below.
8	What does `systemctl` do?	A → C	Manages **services**: `start` , `stop` , `enable` , `disable` , `status` , `reload`

🔍 CLARIFIED

Clarified, since the live answer to #7 was tangled: `man <cmd>` opens the full manual page for a **program on disk** (`man ls` , `man read`). `help <cmd>` gives usage for a **Bash built-in** — commands that live inside the shell itself with no file of their own (`help cd` , `help read`). Rule of thumb: if `man` says "no manual entry", it's a built-in, so try `help` . Most commands also accept `--flag` -style `--help` for a quick summary.

🎮 Her own bonus question: decode `754`

Digit	Who	Permission
7	owner	read + write + execute (4+2+1)
5	group	read + execute (4+1)
4	others	read only

Then the follow-up that mattered: *"Is `754` a good approach or not? Are we using it or not?"*

The room split. Her answer:

✅ LOCK THIS IN

`754` **is the sensible default** — *"most of the 95%, or you can say 99% of cases, we are using these kinds of numbers like `754` : the owner has all permissions to read, write and execute; the group can read and execute; and others can only read. This is a good approach."*

⚠️ **The exception:** *"If you want to do it with the temp folder — then we're not going to use `754` , we're going to use `777` ."* (Same `/tmp` rule from Class 2.)

◆ ◆ ◆

04 🐚 Your First Shell Script — `hello.sh`

The task, framed as a real DevOps task

🎤 SAID IN CLASS

"The task is: we have to print the content of a log file from a particular folder."

Broken into **4 steps**:

```
1. create a directory
2. create a file inside it
3. add some text to that file
4. print that text back out
```

💡 GOOD TO KNOW

Vocabulary note: she said "repository" and immediately corrected herself — *"directory, sorry. Directory, we call it in Linux."* Small thing, but it's the right instinct: in Linux it's a **directory**, not a folder or a repo.

🎯 Interview question #1: how many ways can you create a file?

She keeps asking this on purpose — *"it's basically an interview question, that's why I'm just asking, keeping reminding you."*

#	Way
1	`touch app.log`
2	`vi app.log` / `vim app.log`
3	`nano app.log`
4	`echo "text" > app.log`
5	`cat > app.log` (redirect into a new file)

🎯 Interview question #2: how many ways to add content to a file?

#	Way
1	`echo "this is my log" >> app.log` ✅ *"this is the correct way"*
2	An **editor** — `vi`, `vim`, `nano`
3	Copy-paste into it, or via `cat` / `cp`

The one-step vs two-step debate

```
# Two steps
touch app.log
echo "hello world" >> app.log

# One step — echo creates the file if it doesn't exist
echo "hello world" > app.log
```

🎤 SAID IN CLASS

"Which one will you choose? In programming, you know, we can't add lots of lines." The room said the shorter one — correct instinct.

The live walkthrough

```
mkdir hello
cd hello
touch app.log
echo "This is my log file" > app.log
cat app.log
```

Actual output (and this file is still in the repo at `Class-03/hello/app.log`):

```
This is my log file
```

The five steps she made the class recite back: `mkdir hello` → `cd hello` → `touch` → `echo` → `cat`.

Turning it into a script

🎤 SAID IN CLASS

"So why do we need to use a shell script instead of writing these many commands to perform this single task?" → because on a real server you'd be doing this across many folders, on a schedule, forever.

She copied all five commands into one file: **`hello.sh`**.

🏃 Two ways to run a shell script

```
./hello.sh        # way 1
sh hello.sh       # way 2
```

⚠️ The first run fails

```
permission denied
```

Why: a new file has no **execute** permission. It's text, not a program — yet.

The fix:

```
chmod +x hello.sh
```

Symbolic permissions — the other `chmod` syntax

You already know numeric (`754`). This is the **symbolic** form:

Symbol	Meaning
`+`	**add** a permission
`-`	**remove** a permission
`r`	read
`w`	write
`x`	execute

```
chmod +x file      # add execute
chmod +r file      # add read
chmod -r file      # remove read
chmod +w -r file   # add write, remove read
```

😄 **CLASS MOMENT**

Her trick question: *"He wants to give permission for write, and he wants to remove permission for read. Is that really okay? Are you going to do it?"* Room: *"No, not possible."*

🔍 **Clarified, because the live answer was ambiguous:** it absolutely **is** possible — each flag is independent, so `chmod +w -r file` is valid and will run. The real point she was making is that it's a *bad idea*: a file you can write to but not read is almost never what you want. **Possible ≠ sensible.**

💡 The green-file trick

💡 GOOD TO KNOW

A free visual cue. On Ubuntu-style terminals, run `ls` before and after `chmod +x`:

```
before:  hello.sh   ← white / default colour
after:   hello.sh   ← GREEN
```

"Green means you are giving executable permission to that file — anyone can execute that file." She noted her Mac terminal didn't show the colour change, so try it on an Ubuntu VM or your EC2 instance.

Running it for real

```
./hello.sh
# → This is my log file
```

And the `hello/` folder now exists with `app.log` inside it, containing that one line. First script: done.

😄 CLASS MOMENT

Chaos moment: she accidentally deleted the folder into the **Trash** and then tried to run the script from inside Trash — *"oh, we are in the trash folder, really? Sorry guys, I have to use a new folder."* Also: *"I know how many laptops I am switching, so Control — you have to find where the Control is, if it's a Mac, if it's Windows, I just forget."*

◆ ◆ ◆

05 📝 Assignment 1 — Overwrite AND Append

The task

📝 **Same script, but the output must have TWO lines.** Write one line into the file, then add a second line to the *same* file, and `cat` must print **both**.

"One is once you enter the first sentence in that file. And second is once you overwrite that file content — that overwritten content also I want. Two lines I want in output."

Deadline: 10:40. She checked in repeatedly — *"How many of you are done? Still few. Team C? Where did you get stuck? Tell me, I'll help you — or your friends are not giving you the answer, not showing the screen?"* 😄

💡 GOOD TO KNOW

Her reassurance: *"90% of the code is the same, only one or two lines maybe you change. This is just exercise for your brain to start working with this shell scripting stuff."*

The solution the class built live

```
mkdir test
cd test
echo "this is file 1" > app.log      # > creates and OVERWRITES
echo "this is file 2" >> app.log     # >> APPENDS
cat app.log
```

Output:

```
this is file 1
this is file 2
```

💡 The optimisation she pushed for

🎤 SAID IN CLASS

"Anyone do something else instead of doing `touch` and then `echo`? Exactly — we don't need this `touch` command. I am just reducing the lines."

Because `echo "..." > file` **creates** the file, the separate `touch` is dead weight.

⚠️ The bug she caught in the live run

The first run printed **three** lines instead of two: `this is file 1`, `this is file 1`, `this is file 2`.

Why? She'd left an extra `cat` (or an extra echo) in from the previous version. *"But I want only two lines. Then which one do you want to remove? This one. Okay, I'll remove the test folder. We'll run this. Good. Good exercise."*

✅ LOCK THIS IN

The lesson hiding in the bug: delete the output folder between runs. A script that does `mkdir test` will fail (or double up) on a folder that already exists from the last attempt.

🗂️ The versions of this exercise that exist in the repo

Your file, `Class-03/activity.sh`:

```
# create folder, file, cat with overridden content
# 2 outputs: one: 1st file, as well as overridden content also shd be shown

mkdir direc
cd direc
touch oldfile.log
echo this is old > oldfile.log
echo this is overridden content >> oldfile.log
cat oldfile.log
```

Output in `Class-03/direc/oldfile.log`:

```
this is old
this is overridden content
```

✅ Correct. (It keeps the `touch`, which is the un-optimised version — harmless.)

The instructor's `script1.sh`:

```
mkdir test
cd test
echo "this is file1" > app.log
echo "this is file2" >> app.log
cat app.log
```

The instructor's `data.sh` — a sneaky variant that proves what `>` does:

```
mkdir data1
cd data1
echo "This is a log file." > app.log
cat app.log                       # → This is a log file.
echo "This is my file" > app.log # single > again = WIPES it
cat app.log                       # → This is my file
```

⚠️ WATCH OUT

Read that last one carefully. The second `>` **destroyed** the first line. The file ends up containing only `This is my file` — which is exactly what's in the repo. That's the whole difference between `>` and `>>`, demonstrated in six lines.

◆ ◆ ◆

06 🔤 Variables

First, the shebang: `#!/bin/bash`

She wrote one line at the top of the file and asked what it meant. Nobody had a full answer, so here it is properly.

🧠 PICTURE THIS

Picture this. Bash is just a program on your computer — same kind of thing as Chrome, it simply has no icon. Its job is to read words like `mkdir` and actually do them.

`/bin/bash` is nothing fancier than **where that program lives on disk** — a street address, but for a program instead of a house. (Check it yourself: `which bash`.)

So `#!/bin/bash` on line 1 is a **sticky note on top of the recipe** saying: *"this is written in Bash — use the Bash program at `/bin/bash` to read it."* When you run `./hello.sh`, the computer peeks at that first line, finds the program, and hands it the rest of the file.

The specifics from class:

Piece	What it is
`#!`	"hash-bang" / **shebang** — tells the OS which **interpreter** to use
interpreter	a program that reads code and runs it **line by line, in order**
`/bin/bash`	the **location** of the Bash binary

💡 GOOD TO KNOW

Why bother, if it works without it? Because different distributions ship different shells. *"If you're using CentOS, Fedora, or Red Hat Linux, multiple types of shell are there."* Naming `/bin/bash` explicitly means your script behaves the same on a server you've never seen.

Also: if you run `sh hello.sh` or `bash hello.sh`, you've *already* told the terminal which interpreter to use, so the shebang gets skipped. But `./hello.sh` is the more common way — so always include it.

🔍 **One clarification on the class explanation:** she compared the interpreter to *"a C language interpreter"*. C is normally **compiled**, not interpreted — so the useful half of that comparison is just "something reads the code in order", not a literal C parallel. Bash's real cousin here is **Python**, which she also mentioned.

📌 NOTE

A note found in your own `Class-03/direc/oldfile.log` : the line *"didnt get bin bash"* was appended to the log file during class. Hopefully the explanation above closes that loop. 🙂

The syntax

```
variable="Hey Anishka"
name="Anishka"
echo $variable
echo $name
```

Rule	Detail
Assign	`variable=value`
Read/print	`$variable`

⚠️ The #1 beginner error

```
variable = "hello"      # ❌ BREAKS
variable="hello"        # ✅ works
```

⚠️ WATCH OUT

No spaces around the `=`. *"If you do the same thing — variable, space, equals, space — you will face an error. Some editors support this, but 95% of editors are not supporting this."*

⚠️ Reserved names — you can't name a variable after a command

```
ls="myfile"       # ❌ can't do this — ls is a command
```

> ✅ **LOCK THIS IN**
>
> *"Same as what you have in programming with Java or Python — you cannot use the keywords of the programming language as a variable."*

😄 VS Code outed her GitHub account

She started typing the `name` variable and VS Code autocompleted her real name:

> 🎤 **SAID IN CLASS**
>
> *"Oh my god. This knows how… my name. Oh my god. Okay, okay — I just logged in through GitHub. That's it. From GitHub it's taking my name."*

📝 Task: three variables

> 📝 **Create `variable.sh` with three variables — your name, your roll number, and a comment about the class — and print all three.** *"A very very simple task, you can do it within maybe two minutes."*

The instructor's version:

```
#!/bin/bash

variable="Hello, World!"
echo $variable

#ls="myfile" -- wrong way
#we can't use commands names as a variable name

name="Nensi"
roll_no=123
comment="Awesome"
echo "My name is $name"
echo "My roll number is $roll_no"
echo "I am $comment"
```

💡 GOOD TO KNOW

Note she **kept the wrong-way line as a comment** in her own file — a deliberate reminder of the reserved-name rule.

Why variables matter for automation

✅ LOCK THIS IN

"To do some automation stuff you need to write a script. Every time you can't go through those particular commands and get your output. You need variables, you need loops, you need if statements, you need functions — almost every programming concept — to do some particular automation task."

💡 **And when Bash isn't enough:** *"Some AI stuff they are also using, but at that time they are not using `sh` — they are using Python."* Bash was described as *"almost nearly equal to Python"* for this kind of work.

◆ ◆ ◆

07 ⌨️ Runtime Input — `read -p`

The idea

🧠 PICTURE THIS

Picture this: so far your script *knows* the answers because you typed them into the file. `read -p` makes the script **stop and ask you** while it's running — like a form field instead of a printed page.

The syntax

```
read -p "Enter your name: " name
#      └─ the prompt shown ─┘   └─ the variable it fills
```

Live demo

```
read -p "Enter your name " name
read -p "Enter your roll number " roll_number
echo "My name is $name and my roll number is $roll_number"
```

She ran it with her own values:

```
Enter your name: Nancy
Enter your roll number: 028
→ My name is Nancy and my roll number is 028
```

💡 GOOD TO KNOW

"In college my last-digit roll number was 028."

📌 NOTE

What `-p` actually is: a **flag** on the `read` command that means "show this prompt". Her note: *"It will do the read and then the line — it will not take the variable after the line. It's a flag. You can check the flags with `man read` ."* (She tried to show `man read` live but her terminal font was too large: *"my terminal is too big, I cannot see anything… forget about it, this one we can check only on the server."*)

🗂️ Your version — `Class-03/variable.sh`

```sh
# variable="Hey Anishka"
# name="Anishka"
# echo $variable
# echo $name

read -p "Enter name " name
read -p "Enter roll " roll

echo "My name is $name"
echo "My roll is $roll"
```

✅ Correct — and it usefully keeps the hardcoded-variable version commented out above, so you can see both approaches side by side.

The instructor's `input.sh`

```sh
#!/bin/bash

read -p "Enter your name: " name
read -p "Enter your roll number: " roll_no

echo  "My name is $name"
echo "My roll number is $roll_no"
```

📝 Task: add a third input

> 📝 **Same script, but three inputs — name, roll number, and a comment.** *"I will not give you the script. You can see the script and you need to enter a third one: comment."*

◆ ◆ ◆

08 🚦 Conditions — `if` / `elif` / `else`

> 📌 **NOTE**
>
> **Sourcing note:** the recording cuts away right after she announces *"now we'll do the condition, then the for loop, then the while loop, and last one we'll do the function"* — the audio jumps straight to the combined assignment. §8–§10 come from **her own scripts** in the official course repo (`session3-shell-scripting/`), which is where she told the class the material would live. It's genuinely what was taught that day; it just isn't in the audio.

Her script — `condition.sh`

```bash
#!/bin/bash

read -p "Enter your age: " age

if [ $age -lt 0 ]; then
    echo "Invalid age. Please enter a valid age."
elif [ $age -lt 13 ]; then
    echo "You are a child."
elif [ $age -lt 20 ]; then
    echo "You are a teenager."
else
    echo "You are an adult."
fi
```

Decoding it

Piece	Meaning
`if [ ... ]; then`	start the test
`elif [ ... ]; then`	"else, if this instead"
`else`	the fallback
`fi`	**closes** the `if` block — it's `if` spelled backwards 🙂

Piece	Meaning
`[ ... ]`	the **test** brackets. Spaces inside them are **required**

💡 GOOD TO KNOW

The invisible thing every `if` is actually testing: the exit code.

Every command you run leaves behind a number. `0` **means success**; anything else means a failure. Check it with `$?`:

```
ls /home
echo $?      # → 0    ✅ worked

ls /nope
echo $?      # → 2    ❌ failed
```

⚠️ **Note it's backwards from most languages:** in Bash, **0 is good**. (Because there's only one way to succeed, but many different ways to fail — so the non-zero numbers tell you *which* failure.)

✅ **And this is what unlocks the two operators you'll use constantly:**

```
mkdir build && cd build     # && = run the second ONLY IF the first succeeded
ping -c1 google.com || echo "no internet"   # || = run the second ONLY IF the first FAILED
```

💡 `[ $age -lt 0 ]` is itself just a command that exits `0` when the test is true. That's why `if` works on it — `if` doesn't understand "true", it understands "exited with 0".

The comparison operators

Operator	Means
`-lt`	**l**ess **t**han
`-gt`	**g**reater **t**han
`-le`	less than or equal
`-ge`	greater than or equal
`-eq`	**eq**ual
`-ne`	**n**ot **e**qual

> 💡 **GOOD TO KNOW**
>
> **Why this exact shape matters in real DevOps:** the "sort a number into bands" pattern is everywhere. Swap `age` for disk usage from `df -h` and you have a monitoring script: `< 70%` = fine, `< 90%` = warning, `else` = page someone at 3 AM.

> ⚠️ **WATCH OUT**
>
> **Notice the order.** The `-lt 0` check comes **first**. If you put `-lt 13` first, an age of `-5` would be classified as "a child" — because -5 *is* less than 13. In an if/elif chain, **the first match wins**, so always test the narrowest/most specific case first.

◆ ◆ ◆

09 🔁 Loops

For loop — two shapes

Shape 1 — the range (`{1..5}`) — her `loop.sh`:

```bash
#!/bin/bash
# for in in 1 2 3 4 5

for i in {1..5}
do
  echo "This is iteration number $i"
done
```

Shape 2 — the C-style counter — this one's in **your** `Class-03/loop.sh`:

```bash
#!/bin/bash

for ((i = 0; i<5; i++))
do
    echo "this is iteration number $i"
done
```

⚠️ A correction to the comment in your file

Your `loop.sh` notes *"# prints 1 to 4"* for the C-style loop. It actually prints **0 to 4** — the counter starts at `i = 0`, and stops before 5. Five iterations either way, but the numbers printed are `0 1 2 3 4`.

Loop	Prints
`for ((i = 0; i<5; i++))`	**0, 1, 2, 3, 4**
`for i in {1..5}`	**1, 2, 3, 4, 5**

✅ **LOCK THIS IN**

Both run 5 times. `{1..5}` is **inclusive on both ends**; the C-style version starts where you tell it and stops *before* the limit.

💎 The best note in your file

Your `loop.sh` ends with this, and it's genuinely the thing that trips everyone up:

```
# []   => is a condition/test
# (()) => tells bash that something written inside this should be
#         treated as math/arithmetic
```

✅ **LOCK THIS IN**

Lock this in. Square brackets = *"is this true?"* · Double parentheses = *"do maths in here."* That's why you write `[ $count -lt 5 ]` for the test but `((count++))` for the increment.

While loop #1 — counter-based (`while_loop1.sh`)

```
#!/bin/bash

count=0
while [ $count -lt 5 ]
do
  echo "This is iteration number $count"
  ((count++))
done
```

How it reads: *"while count is less than 5: print it, then add one."* Forget the `((count++))` and you have an infinite loop.

While loop #2 — interactive with validation (`while_loop.sh`)

This one bundles everything from today into one script:

```bash
#!/bin/bash

while true; do
    read -p "Enter a number (or 'q' to quit): " input

    if [[ $input == "q" ]]; then
        echo "Exiting the loop."
        break
    elif ! [[ $input =~ ^[0-9]+$ ]]; then
        echo "Invalid input. Please enter a valid number."
        continue
    fi

    echo "You entered: $input"
done
```

🏴 Your own explanation of it — `Class-03/while.md`

🎤 SAID IN CLASS

"This while loop will just take an input and check if it's a number. If it's not a number, it will give an error. It's an infinity loop which will break when the input is `q` . And if it's a number, then it will just return that number and continue the next iteration."

✅ **That's a completely correct reading of the script.** Here's the machinery behind it:

Piece	What it does
`while true; do`	a **deliberate infinite loop** — you must exit it manually
`break`	leave the loop **entirely**
`continue`	skip the rest of this round, go **straight to the next** iteration
`[[ ... ]]`	double brackets — a **more powerful** test than `[ ... ]`
`=~`	**regex match** — only works inside `[[ ]]`, not `[ ]`
`^[0-9]+$`	"start-to-end, one or more digits" → i.e. **digits only**
`!`	**not** — so `! [[ ... ]]` means "if it does NOT match"

✅ LOCK THIS IN

Why the `[[ ]]` vs `[ ]` distinction is worth remembering: regex matching with `=~` is exactly the thing single brackets **can't** do. When you need to validate a format — a number, an IP, a filename pattern — reach for `[[ ]]` .

◆ ◆ ◆

10 🧩 Functions

Her script — `function.sh`

```
#!/bin/bash

show_info(){
  echo "This is a function"
  echo "This is a function to show information"
}

show_info()
```

The shape

```
function_name(){
    ...body...
}
```

⚠️ There's a bug in that last line

⚠️ WATCH OUT

`show_info()` **on its own will error.** `name()` followed by `{ }` is how you **define** a function. To **call** one, you write its bare name with **no parentheses**:

```
show_info       # ✅ this calls it
show_info()     # ❌ this looks like the start of another definition
```

💡 GOOD TO KNOW

The one scripting piece the class list mentions but never reaches: arguments.

Right now your scripts either hardcode values or ask with `read -p`. **Arguments** let you pass them in when you run the script — which is what you need for anything a cron job will run, since nobody's there to type an answer.

```
#!/bin/bash
echo "Backing up $1 into $2"
echo "You passed $# arguments: $@"
```

```
./backup.sh /var/log /mnt/backups
# → Backing up /var/log into /mnt/backups
# → You passed 2 arguments: /var/log /mnt/backups
```

Variable	Means
`$0`	the script's own name
`$1`, `$2`, `$3` …	the 1st, 2nd, 3rd argument
`$@`	**all** the arguments
`$#`	**how many** arguments

✅ **The same numbering works inside a function**, which is what makes functions genuinely reusable:

```
greet() {
    echo "Hello, $1!"
}

greet Anishka       # → Hello, Anishka!
greet Nensi         # → Hello, Nensi!
```

✅ Your version got it right — `Class-03/function.sh`

```
hello() {
    echo "Anishka"
}

hello
```

Define with `()`, call without. That's the correct pattern — your file is the one to copy from.

◆ ◆ ◆

11 📝 The Big Combined Assignment

She dropped this into the **WhatsApp group** as `task.md`. It combines everything from the day.

The assignment sheet, verbatim

```
# print current date - date
# hostname and username - hostname whoami who w
# process ps
# add process info inside a file name process.log --> > process.log

# print name, roll_no, comment

## use variables, take input, create file and directory
```

The requirements, spelled out

#	Requirement	Hint she gave
1	Print **name**, **roll number**, **comment**	must use **variables** captured with `read -p`
2	Print the **current date**	🎯 *"There is a command `date`. But you have to take it as input. This is the thing you need to think about — how you can take the input from that same command and then print it."*
3	Print the **hostname**	
4	Print the **username**	*"the simplest thing"*
5	Print **disk usage**	`df -h`
6	List **processes** and save them **into a file**	`ps > process.log` — inside a **newly created directory**

Deadline: 11:15. (She gave a few extra minutes after the first check-in.)

🎓 Her method for writing any script — worth stealing

✅ LOCK THIS IN

This is the actual transferable skill of the class:

```
1. Split the task into sub-tasks.  "task one, task two, task three"
2. For each sub-task, figure out WHICH command does it.
3. RUN that command on its own. Confirm it gives the output you want.
4. Only THEN assemble the confirmed commands into the script.
```

"Then I get to know — okay, now I can assemble those commands, and I just put it inside a script. That's a simple way."

⚠️ WATCH OUT

Her explicit warning: *"Don't directly copy-paste my thing from WhatsApp into ChatGPT. I'm telling you — don't directly do it. Otherwise your mind will not work when the interviewer asks you something, or after you are working in a company. Just try to think which command you can use."*

😄 Her WhatsApp-reply confession

😄 CLASS MOMENT

"How many of you are done? You can reply to that message that 'done'. I used to do it when my professor pinged us in a WhatsApp chat — I would either ping at last or ping at first. That's it. Because they need to notice me. That's all."

🗂️ Your submitted attempt — `Class-03/task.sh`

```
# print current data
# hostname and username
# process
# add process info inside a file name process.log
# print name,roll_no, comment
## use variables, take input, create file and directory

mkdir task
cd task
touch result.log
echo "This is my result file" > result.log
date
echo $hostname
echo $whoami
df -h

# read -p "Mention current data" date
# echo "Date is $date"

ps > process.txt
# echo current_date=$date >> result.log - wont give date output
echo current_date=$(date) >> result.log

read -p "Username: " name
echo "My name is $name" >> result.log
```

What this produced — `Class-03/task/result.log`:

```
This is my result file
current_date=
My name is Anishka
```

And `Class-03/task/process.txt` captured the real process list (`PID TTY TIME CMD`, several `-zsh` entries, and `sh ./task.sh` — the script catching itself running 🙂).

🐛 The two bugs in it, and they're the two most instructive bugs of the class

Bug 1 — `echo $hostname` and `echo $whoami` print nothing.

⚠️ WATCH OUT

Why: `hostname` and `whoami` are **commands**, but `$hostname` and `$whoami` ask for **variables** with those names — and no such variables exist, so Bash substitutes empty strings.

```
echo $hostname       # ❌ prints a blank line
hostname             # ✅ runs the command
host=$(hostname)     # ✅ captures its output into a variable
echo $host           # ✅ prints it
```

Bug 2 — `current_date=` came out empty in `result.log`.

Look at the line that produced it:

```
echo current_date=$(date) >> result.log
```

The `$(date)` part is correct. But there are **no quotes**, so `date`'s output — which contains spaces — got split into separate words by the shell. `echo` printed `current_date=` plus the date as extra arguments, and the redirect captured a mangled result.

✅ LOCK THIS IN

The fix — quote it:

```
current_date=$(date)                        # capture first
echo "current_date=$current_date" >> result.log   # then print, quoted
```

Two habits in one: **assign, then use**, and **quote anything containing spaces**.

✅ LOCK THIS IN

Credit where due: the commented-out line in your file — `# echo current_date=$date >> result.log - wont give date output` — shows you'd already spotted that `$date` alone doesn't work and reached for `$(date)`. That's exactly the right diagnosis; it just needed the quotes too.

🔑 The concept both bugs point at: command substitution

✅ LOCK THIS IN

`$(command)` — **run the command, hand me its output.** This is the answer to her *"take the date as input"* hint.

```
current_date=$(date)      # ✅
user=$(whoami)            # ✅
host=$(hostname)          # ✅
```

The instructor's own answer — `task.md`'s script section

```
current_date=$(date)
echo $current_date

echo $hostname
echo $whoami
ps > process.log
read -p "Enter your name: " name
read -p "Enter your roll number: " roll_no
read -p "Enter your comment: " comment

echo  "My name is $name"
echo "My roll number is $roll_no"
echo "My comment is: $comment"
```

😄 CLASS MOMENT

Notice: even the instructor's own version has the `echo $hostname` / `echo $whoami` bug in it. She wrote the correct fix for `date` (`$(date)`) but not for the other two. So if you made that mistake — you're in good company. Now you know the fix and she… also knows. 🙂

Her fuller version, `test1.sh`:

```bash
#!/bin/bash

mkdir result_file
cd result_file
touch result.log
echo "This is my result file" > result.log
date
echo $hostname
echo $whoami
df -h
ps > process.log

read -p "Enter your name: " name
read -p "Enter your roll number: " roll_no

current_date=$(date)
echo "My name is $name" >> result.log
```

◆ ◆ ◆

12 📜 Live Review of Submitted Scripts

She went round the room asking what each script contained.

🎤 SAID IN CLASS

The question she pressed hardest on: *"Did you actually run those commands, and after testing them it gave your output, and then you put those lines inside the shell script? Are you doing this?"*

Room, cheerfully: *"We are genius, we will do directly."* 😄

✅ The final checklist she read out

Every script was expected to use all of these:

- ✓ `cd`
- ✓ `mkdir`
- ✓ `touch`
- ✓ `echo`
- ✓ `df -h`
- ✓ `ps`
- ✓ `>` (redirection)
- ✓ `cat`
- ✓ `read -p`

✓ the current date **captured into a variable**

Show of hands results: most had almost everything. *"Who missed one thing? Like `read -p`, or maybe some variable, or maybe the `date` command? Almost close. Good, good going. Two things? What about others — they missed the complete shell script? Really?"* 😄

> ⚠️ **WATCH OUT**
>
> **The most-missed requirement: the `read -p` / variable part.** Several students hardcoded the values or ran the commands directly instead of storing results in variables. If you're revising, that's the bit to redo.

> 💡 **GOOD TO KNOW**
>
> **She also accepted alternatives:** for the process task, *"`ps` is fine, you can do `echo` also, you can use the `grep` command also, you can use the pipe command also."* Multiple valid answers — the Class 2 philosophy again.

◆ ◆ ◆

13 👋 Wrap-Up

📤 The GitHub instruction

📝 **Right now, in class:** create a GitHub repo and push **all four scripts** from today:

1. the `hello`/log demo
2. the overwrite/append task
3. the variable + input task
4. the combined `task.sh`

Then ping the public repo link. *"Once you complete pushing this code, let me know — I want to see."*

Why she wanted it live: *"So I get to know how many people are actually doing it and how many are not, so I can send it to them, they can review, they can check on you."*

She also floated sharing a **CSV sheet** for tracking completion — then immediately worried aloud: *"someone will remove someone's name."* 😄

🗒️ Attendance

Taken by roll number, students calling out their last digits: 4, 10065, 10198, 10622, 354, 201, 1066, 330…

😄 CLASS MOMENT

She got stuck on one name — *"Arka Ghosh. Arka Ghosh. I don't know how to pronounce it. Is he an instructor?"* — then: *"Sorry, I will put my accent up."*

🎓 Her own college story

The class got asked whether they get assignments every term. Then she shared hers:

😄 CLASS MOMENT

"Because I studied in a government college — you have to take a file, you have to put two pages, one page for the index, one page that the professor told us to make, and they need the assignment in written notes. And some people were like — okay, do the assignment, create a PDF, print out, get it here, that's it. They'll just sign the last page. They'll not see your assignment, they'll not see your page, nothing. Just — 'last three digits?' '028.' 'Okay, 028, she's there' — then they sign it and put my marks."

"It was a good time, by the way."

She's from a **Government Engineering College in Gujarat**, under **GTU** (Gujarat Technological University). *(The college's town name comes through garbled in the recording.)*

✅ LOCK THIS IN

And her point in telling it: *"This is a good class — when I'm doing and you are also doing."* The contrast is deliberate: here the work actually gets read.

She also asked who the class **topper** was. 😄

◆ ◆ ◆

14 📔 Cheat-Sheet — Everything From Today

Script mechanics

Thing	Does
`#!/bin/bash`	**shebang** — which interpreter runs this script
`./script.sh`	run it (needs `chmod +x` first)
`sh script.sh`	run it (shebang is skipped)
`bash script.sh`	same idea
`chmod +x script.sh`	make it executable
`which bash`	find where Bash actually lives

Thing	Does
`# comment`	a comment

Permissions (symbolic form)

Command	Does
`chmod +x f`	add execute
`chmod +r f` / `+w f`	add read / write
`chmod -r f`	remove read
`chmod 754 f`	owner `rwx`, group `r-x`, others `r--` ← the sensible default
`chmod 777 f`	everything for everyone ← only for `/tmp`-style cases

Variables & input

Thing	Does
`var=value`	assign — **no spaces around** `=`
`$var`	read the value
`read -p "prompt " var`	ask the user at runtime
`$(command)`	⭐ **command substitution** — capture a command's output
`man read`	see a command's flags

Conditions

Thing	Does
`if [ ... ]; then` … `fi`	the block (always closed with `fi`)
`elif` / `else`	extra branches
`[ ... ]`	test brackets — **spaces required inside**
`[[ ... ]]`	enhanced test — supports `=~` regex
`-lt -gt -le -ge -eq -ne`	numeric comparisons
`==`	string equality (inside `[[ ]]`)
`!`	not
`=~ ^[0-9]+$`	regex: digits only

Loops

Thing	Does
`for i in {1..5}`	loop over **1** → **5** (inclusive)
`for ((i=0; i<5; i++))`	C-style loop, **0** → **4**
`while [ cond ]; do ... done`	loop while true
`while true; do ... done`	infinite loop — needs `break`
`((count++))`	arithmetic increment
`break`	exit the loop
`continue`	skip to the next iteration
`[ ]` **vs** `(( ))`	**test** vs **maths**

Functions

Thing	Does
`name(){ ... }`	**define** a function
`name`	**call** it — no parentheses

File & system commands used today

Command	Does
`mkdir d`	make a directory
`cd d` / `cd ..`	move around
`touch f`	create an empty file
`echo "x" > f`	write (**overwrite**)
`echo "x" >> f`	**append**
`cat f`	print a file
`ls` / `ll`	list / long-list
`vi` `vim` `nano`	editors
`date`	current date & time
`hostname`	machine name
`whoami` / `who` / `w`	current user
`df -h`	disk usage, human-readable

Command	Does
`ps`	running processes
`ps > process.log`	save the process list to a file
`grep`	search text
`|`	pipe
`man <cmd>`	manual page
`help <cmd>`	help for a **shell built-in**
`systemctl start|stop|enable|disable|status|reload`	manage services
`cp` / `mv`	copy / move & rename

◆ ◆ ◆

15 📝 Homework Round-Up

#	Task
1	**Write all four scripts** and push them to a **public GitHub repo**; share the link
2	Read the **3 PDFs** — basic Linux, advanced Linux, and the cheat sheet
3	Redo the combined task **using `read -p` and variables** if you skipped that part
4	Practise the workflow: **split the task → test each command alone → then assemble**
5	Still carried over from Class 2: **soft link vs hard link**, `journalctl`, and the **user-management** assignment

◆ ◆ ◆

16 🗒️ Notes on the Transcript Itself

- **Instructor name:** the transcript header credits *"Ritesh Prajapati"* — that's the **developer of the Scaler++ Chrome extension** that downloaded the file, not the teacher. The instructor is **Nensi Ravaliya** (GitHub @Nency-Ravaliya, heard as *"Nancy"* in the audio, and she uses `Nensi` in her own scripts). She's a **DevSecOps Engineer at Standard Chartered** — which is the bank in the Class 2 outage story.
- **The "reference repo" is actually hers.** Nency-Ravaliya/devops-heros is the official **"DevOps Heroes"** course repo she promised the batch in Class 2 — one folder per session, `session1` through `session10`. Where the audio drops out (§8–§10), her own scripts from `session3-shell-scripting/` fill the gap.

- **The audio genuinely cuts out** right after she announces conditions → for loop → while loop → functions, jumping straight to the combined assignment. Those four topics were taught; the recording just doesn't contain them.
- The transcript is littered with **stray Ukrainian, Spanish and Hindi fragments** — dubbing/translation artifacts from the transcription tool during the quiet work-time stretches (*"Дякую"* = "thank you", *"Gracias"*). Two carried real content and are folded in above: the **Spanish restatement of the combined-task requirements** (§11) and **Hindi-English quiz answers** (§3).
- Also in the repo but **not from this class**: `session3-shell-scripting/asd.md` contains subnet-mask scratch notes (`255.0.0.0 - class A` , CIDR examples, host vs network IP). That's **Class 4** material — she was clearly drafting the networking session in the same folder.
- Garbled terms corrected from context: **"self-stripping / shell stripping / sheel script" → shell scripting**, **"MKDIR / mtdir" →** `mkdir` , **"bin bash" →** `/bin/bash` , **"read-p / read hyphen p" →** `read -p` , **"ch mode" →** `chmod` , **"Ego" →** `echo` , **"cd test / cab" →** `cd` **/** `cat` , **"GDU / GTO" → GTU**, **"Amnawat" → an unclear Gujarat town name**.